Desafio Técnico – Desenvolvedor Front-end

Objetivo

Construir um front-end em React que permita **autenticar**, **listar/filtrar imóveis**, **ver detalhes**, **favoritar**, e (para corretor/admin) **criar/editar/deletar imóveis**, consumindo os endpoints descritos. **No fim deste documento está a documentação da API, assim como suas rotas.**

Este desafio simula um cenário real de desenvolvimento front-end em um portal imobiliário: um usuário acessa imóveis disponíveis, aplica filtros e paginação, visualiza detalhes e pode favoritar anúncios. Quando autenticado, o usuário também tem acesso à sua área de conta e, dependendo do perfil, pode gerenciar imóveis cadastrados.

Requisitos:

1) Autenticação

- Tela de **Login**.
- Tela de **Register**.
- Persistir token JWT, tokens JWT utilizam HEADER: Authorization: Bearer <token>.
- Proteger rotas que exigem JWT.

2) Área pública: listagem e detalhes

Página Home / Listagem

- Consumir endpoints de PROPERTY com:
  - paginação (page, size)
  - ordenação (sort)
  - filtros:
    - name, type, minPrice, maxPrice, minBedrooms
- Mostrar card/lista com: nome, cidade/estado, tipo, valor, quartos, área, ativo/inativo
- A listagem deve suportar:
  - debounce em busca por nome
  - atualização de URL (querystring) ao aplicar filtros (state ↔ URL)

Página Detalhe

- Consumir endpoint de DETALHES.
- Exibir galeria de imagens**.**

3) Área autenticada: “Minha conta” •

Página “Meu perfil” (getMe),

- Atualizar perfil:
  - nome e senha opcionais
- Página “Favoritos” ○ Listar favoritos.
  - Favoritar. ○ Desfavoritar.

4) Área autenticada: “Minhas propriedades”

- Página “Minhas propriedades”.
- Criar propriedade.
  O upload da imagem dependerá de uma nova integração. Utilize um CDN de sua confiança, pode ser: Cloudinary, Firebase Storage, AWS, etc.
- Editar propriedade.
- Alterar status ativo/inativo.
- Deletar.

Documentação das Rotas da API

Base URL: https://d-engeman.onrender.com

1. Autenticação

Base Path: **/api/auth**

POST /login

Autentica um usuário e retorna o token JWT.

Request Body:

```
{
  "email": "user@example.com",
  "password": "password123"
}
```

Response (200 OK):

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

POST /register

Registra um novo usuário.

Request Body (RegisterDTO):

```
{
  "name": "João Silva", // min 3, max 100 caracteres, não vazio
  "email": "joao@example.com", // formato de e-mail válido, não vazio   "password":
"senhaSegura123" // min 6 caracteres, não vazio
}
```

Response (200 OK).

2. Propriedades

Base Path: **/api/property** GET

/

Lista propriedades com paginação e filtros opcionais.

Query Params (opcionais): name (String), type (Enum: RESIDENCIAL, COMERCIAL, etc), minPrice (Double), maxPrice (Double), minBedrooms (Integer), page (Integer, default 0), size (Integer, default 10), sort (String, default "id").

Response (200 OK):

```
{
  "content": [
    {
      "id": 1,
      "name": "Apartamento no Centro",
      "description": "Lindo apartamento com 3 quartos.",
      "type": "RESIDENCIAL",
      "value": 350000.00,
      "area": 85,
      "bedrooms": 3,
      "address": "Rua das Flores, 123",
      "city": "São Paulo",
```

```
    "state": "SP",
    "active": true,
    "brokerId": 2,
    "brokerName": "Corretor Silva",
    "imageUrls": "https://img1.com,https://img2.com"
  }
 ],
 "pageable": { ... },
 "totalElements": 1,
 "totalPages": 1,
 "size": 10,
 "number": 0
}
```

GET /{id}

Busca os detalhes de uma propriedade específica.

Path Variable: id (Long)

Response (200 OK): mesmo objeto da lista acima. GET

/getUserProperties

Lista todas as propriedades cadastradas pelo corretor logado. Requer Autenticação JWT.

Response (200 OK): lista do mesmo objeto de propriedade. POST /

Cria uma nova propriedade. Requer Autenticação JWT.

Request Body (PropertyCreateDTO):

```
{
 "name": "Casa de Praia", // min 10, max 100 caracteres, não nulo/vazio
 "description": "Excelente vista para o mar.", // não vazio
 "type": "RESIDENCIAL", // não nulo (Enum)
 "value": 500000.00, // não nulo, positivo
 "area": 120, // não nulo, positivo
 "bedrooms": 4, // não nulo, positivo
 "address": "Av. Beira Mar, 100", // não vazio
 "city": "Rio de Janeiro", // não vazio
 "state": "RJ", // não vazio
 "imageUrls": "https://img.com/1.jpg" // não vazio
```

```
}
```

Response (201 Created): objeto da propriedade com ID gerado, igual ao GET. PUT /{id}

Atualiza uma propriedade existente. Requer Autenticação JWT.

Path Variable: id (Long)

Request Body (PropertyUpdateDTO): todos os campos são opcionais.

```
{
  "name": "Casa de Praia Atualizada", // min 10, max 100 caracteres
  "description": "Nova descrição do imóvel.",
  "type": "RESIDENCIAL",
  "value": 480000.00, // positivo
  "area": 125, // positivo
  "bedrooms": 5, // positivo
  "address": "Av. Beira Mar, 102",
  "city": "Rio de Janeiro",
  "state": "RJ",
  "brokerId": 2
}
```

Response (201 Created): objeto da propriedade atualizado.

DELETE /{id}

Deleta uma propriedade. Requer Autenticação JWT e privilégios ou o usuário ser dono da propriedade.

Path Variable: id (Long)

Response: 204 No Content. PATCH

/status/{id}

Altera o status (ativo/inativo) de uma propriedade. Requer Autenticação JWT.

Path Variable: id (Long)

Response (200 OK): objeto da propriedade atualizado com a flag active alternada.

3. Usuário

Base Path: **/api/user** GET

/

Retorna os dados do usuário autenticado (getMe). Requer Autenticação JWT.

Response (200 OK):

```
{
  "id": 1,
  "name": "João Silva",
  "email": "joao@example.com",
  "role": "USER"
}
```

PUT /update

Atualiza os dados do usuário logado. Requer Autenticação JWT.

Request Body (UserUpdateDTO): campos opcionais.

```
{
  "name": "João Santos Silva", // min 3, max 100 caracteres
  "password": "novaSenhaSec" // min 6 caracteres
}
```

Response (201 Created): objeto do usuário atualizado.

POST /create

Cria um novo usuário com função específica. Requer Autenticação JWT e Permissão de ADMIN (hasRole('ADMIN')).

Request Body (UserCreateDTO):

```
{
  "name": "Admin Sistema", // min 3, max 100 caracteres, não vazio
  "email": "admin@example.com", // formato de e-mail válido, não vazio
  "password": "senhaForteAdmin", // min 6 caracteres, não vazio   "role": "ADMIN" // não
nulo (Enum)
}
```

Response (201 Created): objeto do usuário recém-criado. GET /favorites

Lista as propriedades favoritadas pelo usuário logado. Requer Autenticação JWT.

Response (200 OK): lista de propriedades igual ao /api/property.

POST /favorites/{propertyId}

Adiciona uma propriedade aos favoritos do usuário logado. Requer Autenticação JWT.

Path Variable: propertyId (Long) Response:

204 No Content.

DELETE /favorites/{propertyId}

Remove uma propriedade dos favoritos do usuário. Requer Autenticação JWT.

Path Variable: propertyId (Long) Response:

204 No Content.

Observações:
A rota é pública para o escopo de usuário.
Os usuários padrões são:
1. Administrador (Role: ADMIN)

- Permissões: Criar novos usuários (admins/corretores), cadastrar e gerenciar qualquer imóvel.

- E-mail: admin@imobiliaria.com

- Senha: 123456

2. Corretor (Role: CORRETOR)

- Permissões: Cadastrar novos imóveis e gerenciar apenas os imóveis criados por ele.

- E-mail: corretor@imobiliaria.com

- Senha: 123456

3. Cliente (Role: CLIENTE)

- Permissões: Apenas visualizar imóveis, filtrar e favoritar.

- E-mail: cliente@gmail.com

- Senha: 123456